

E-JAVA LAB ASSIGNMENT-5

24/04/2026

1. Write a program to create a Login Form with Username, Password, Login and Clear buttons.

```
import javax.swing.*;
import java.awt.event.*;

public class LoginForm {
    public static void main(String[] args) {
        JFrame f = new JFrame("Login Form");

        JLabel l1 = new JLabel("Username:");
        JLabel l2 = new JLabel("Password:");

        JTextField t1 = new JTextField();
        JPasswordField t2 = new JPasswordField();

        JButton b1 = new JButton("Login");
        JButton b2 = new JButton("Clear");

        l1.setBounds(50, 50, 100, 30);
        l2.setBounds(50, 100, 100, 30);
        t1.setBounds(150, 50, 150, 30);
        t2.setBounds(150, 100, 150, 30);
        b1.setBounds(80, 160, 80, 30);
        b2.setBounds(180, 160, 80, 30);

        f.add(l1); f.add(l2);
        f.add(t1); f.add(t2);
        f.add(b1); f.add(b2);

        // Login button action
        b1.addActionListener(e -> {
            String user = t1.getText();
            String pass = new String(t2.getPassword());

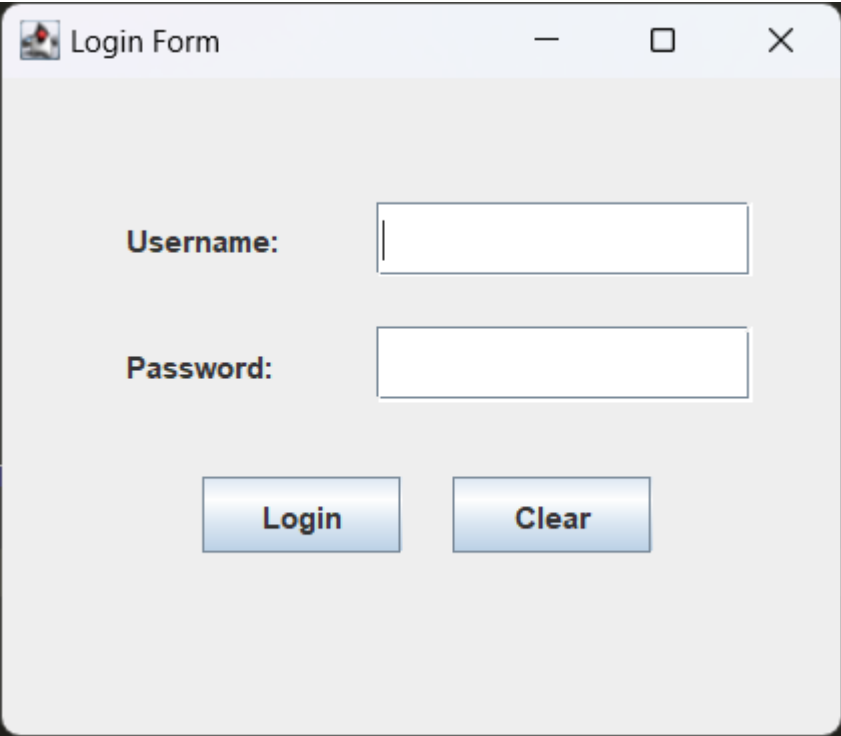
            if (user.equals("admin") && pass.equals("1234"))
                JOptionPane.showMessageDialog(f, "Login Successful");
            else
                JOptionPane.showMessageDialog(f, "Invalid Login");
        });

        // Clear button action
        b2.addActionListener(e -> {
            t1.setText("");
            t2.setText("");
        });

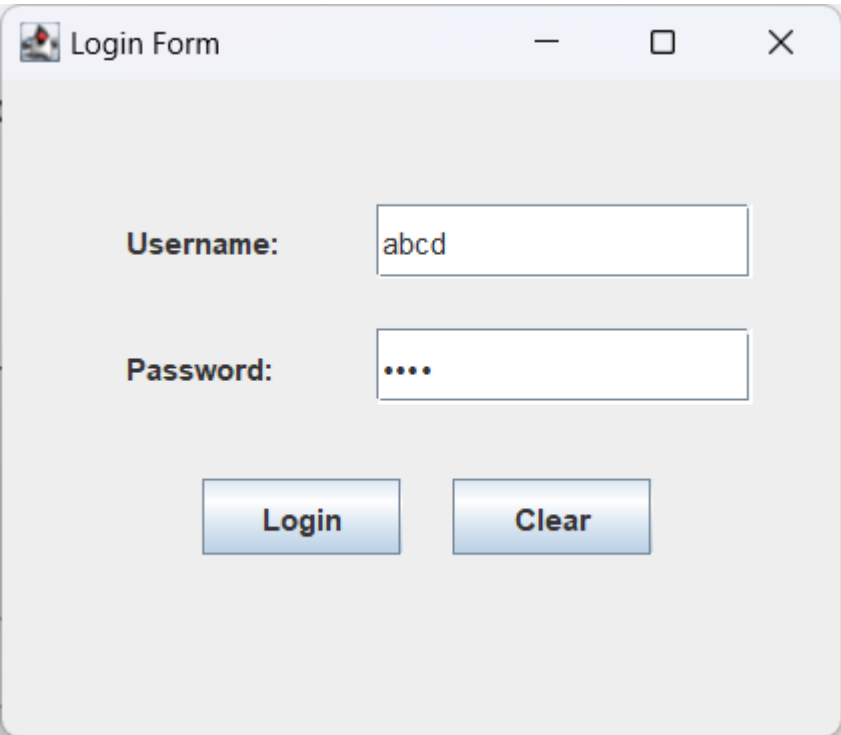
        f.setSize(350, 300);
        f.setLayout(null);
        f.setVisible(true);
        f.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    }
}
```

```
}  
}
```

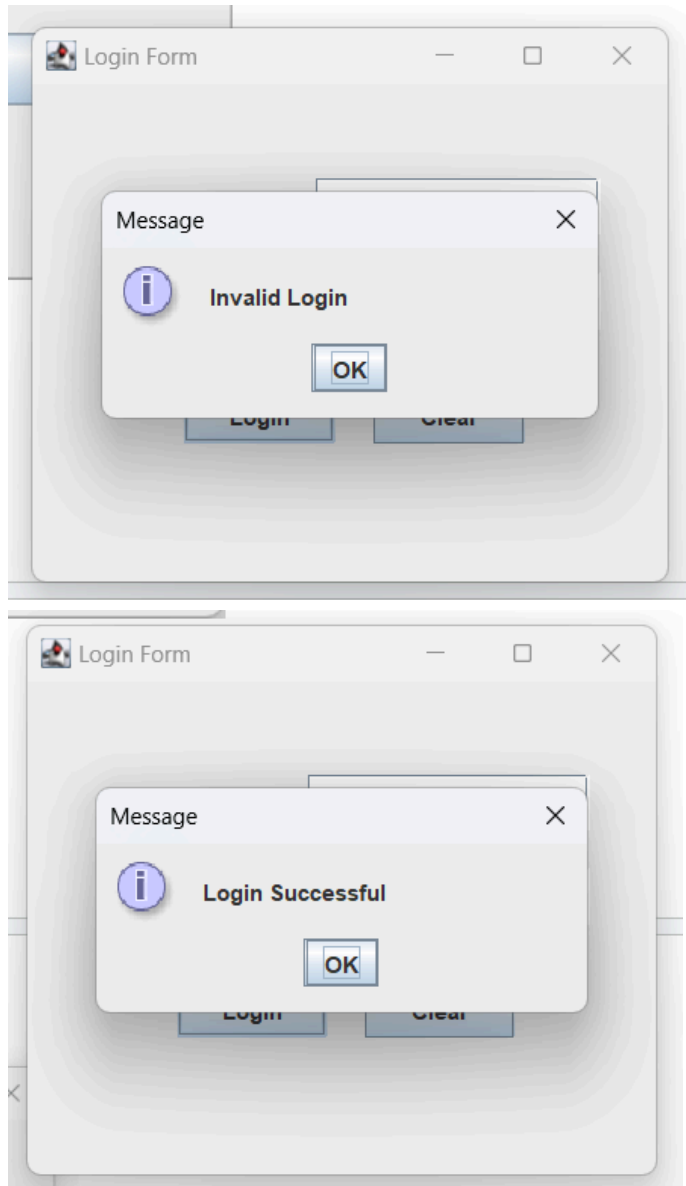
Output:



A screenshot of a Java Swing window titled "Login Form". The window has a standard title bar with a minimize button, a maximize button, and a close button. The main content area is light gray and contains two labels, "Username:" and "Password:", each followed by a white text input field. Below the input fields are two blue buttons with white text: "Login" and "Clear".



A second screenshot of the same "Login Form" window. In this state, the "Username:" input field contains the text "abcd". The "Password:" input field contains four black dots, indicating a masked password. The "Login" and "Clear" buttons remain at the bottom of the form.



2. Write a program to create a Simple Calculator with +, -, ×, ÷ operations.

```
import javax.swing.*;
import java.awt.event.*;

public class Calculator {
    public static void main(String[] args) {

        JFrame f = new JFrame("Simple Calculator");

        JTextField t1 = new JTextField();
        JTextField t2 = new JTextField();
        JTextField res = new JTextField();

        JButton add = new JButton("+");
        JButton sub = new JButton("-");
        JButton mul = new JButton("*");
        JButton div = new JButton("/");

        t1.setBounds(50,50,100,30);
        t2.setBounds(200,50,100,30);
        res.setBounds(120,100,100,30);
```

```

add.setBounds(50,150,50,30);
sub.setBounds(110,150,50,30);
mul.setBounds(170,150,50,30);
div.setBounds(230,150,50,30);

f.add(t1); f.add(t2); f.add(res);
f.add(add); f.add(sub); f.add(mul); f.add(div);

// Actions
add.addActionListener(e -> {
    int a = Integer.parseInt(t1.getText());
    int b = Integer.parseInt(t2.getText());
    res.setText(String.valueOf(a + b));
});

sub.addActionListener(e -> {
    int a = Integer.parseInt(t1.getText());
    int b = Integer.parseInt(t2.getText());
    res.setText(String.valueOf(a - b));
});

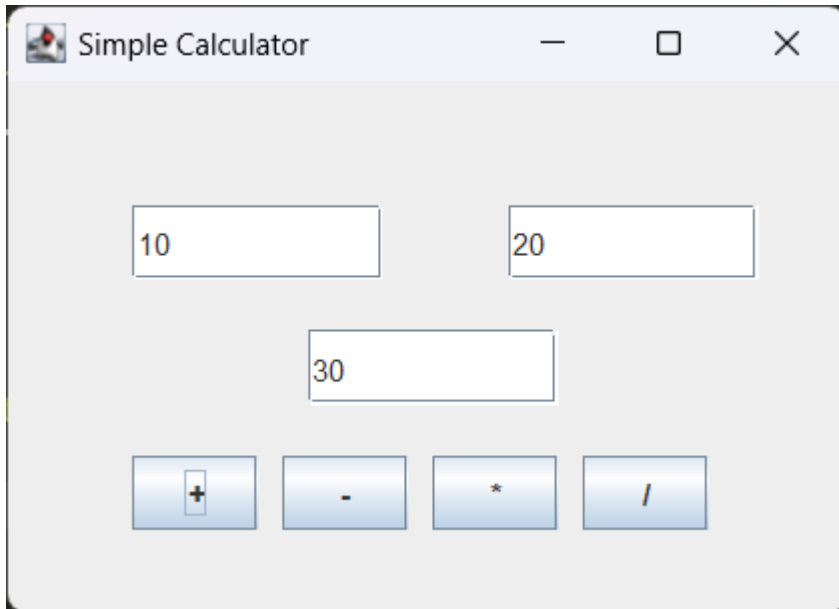
mul.addActionListener(e -> {
    int a = Integer.parseInt(t1.getText());
    int b = Integer.parseInt(t2.getText());
    res.setText(String.valueOf(a * b));
});

div.addActionListener(e -> {
    int a = Integer.parseInt(t1.getText());
    int b = Integer.parseInt(t2.getText());
    res.setText(String.valueOf(a / b));
});

f.setSize(350,250);
f.setLayout(null);
f.setVisible(true);
f.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
}
}

```

Output:



3. Write a program handle keyboard event.

```
import javax.swing.*;
import java.awt.event.*;

public class KeyEventDemo {
    public static void main(String[] args) {

        JFrame f = new JFrame("Keyboard Event");

        JTextField t = new JTextField();
        JLabel l = new JLabel("Press any key...");

        t.setBounds(50,50,200,30);
        l.setBounds(50,100,200,30);

        f.add(t);
        f.add(l);

        t.addKeyListener(new KeyListener() {

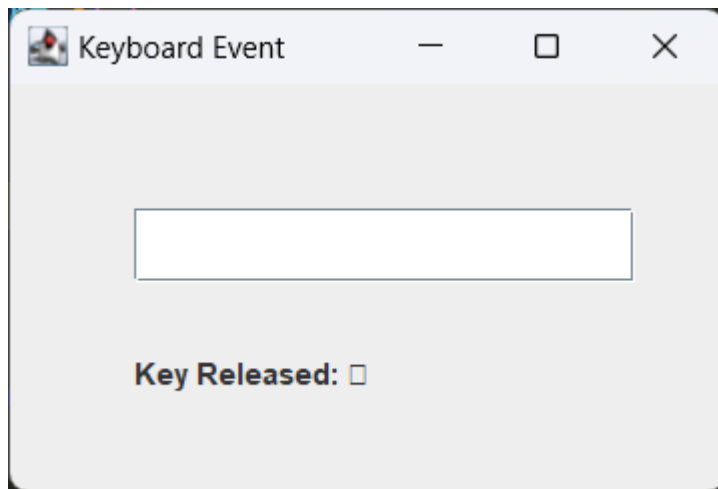
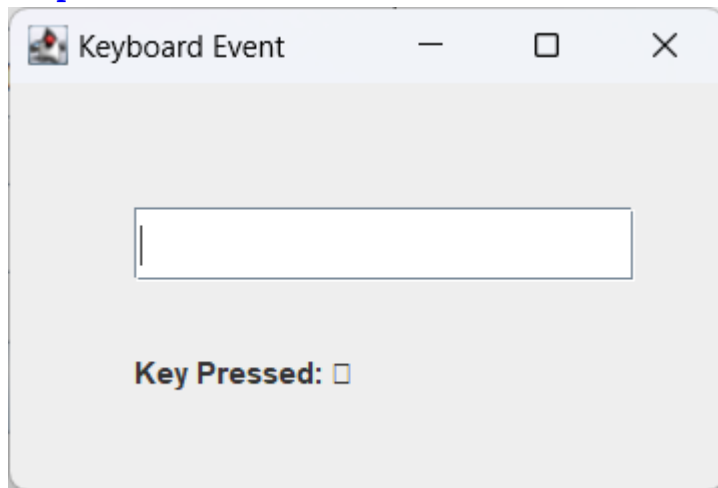
            public void keyPressed(KeyEvent e) {
                l.setText("Key Pressed: " + e.getKeyChar());
            }

            public void keyReleased(KeyEvent e) {
                l.setText("Key Released: " + e.getKeyChar());
            }

            public void keyTyped(KeyEvent e) {
                l.setText("Key Typed: " + e.getKeyChar());
            }
        });

        f.setSize(300,200);
        f.setLayout(null);
        f.setVisible(true);
        f.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    }
}
```

Output:



4. Write a program to implement some mouse events using adapter class.

```
import javax.swing.*;
import java.awt.event.*;

public class MouseEventDemo {
    public static void main(String[] args) {

        JFrame f = new JFrame("Mouse Event");

        JLabel l = new JLabel("Do Mouse Actions");
        l.setBounds(50,50,200,30);

        f.add(l);

        f.addMouseListener(new MouseAdapter() {

            public void mouseClicked(MouseEvent e) {
                l.setText("Mouse Clicked");
            }

            public void mouseEntered(MouseEvent e) {
                l.setText("Mouse Entered");
            }

            public void mouseExited(MouseEvent e) {
                l.setText("Mouse Exited");
            }
        });
    }
}
```

```

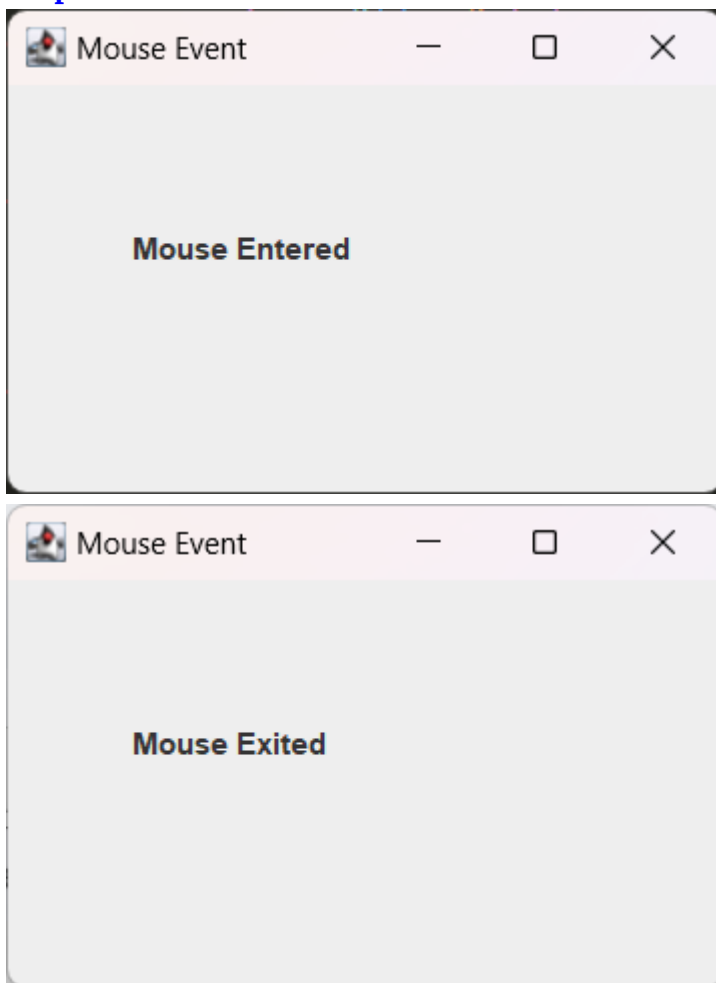
    public void mousePressed(MouseEvent e) {
        l.setText("Mouse Pressed");
    }

    public void mouseReleased(MouseEvent e) {
        l.setText("Mouse Released");
    }
}

f.setSize(300,200);
f.setLayout(null);
f.setVisible(true);
f.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
}
}

```

Output:



5. Write a Java TCP program to simulate a simple Client-Server Chat where client sends a message and server replies.

Server.java:

```

import java.io.*;
import java.net.*;

public class Server {

```

```

public static void main(String[] args) throws Exception {

    ServerSocket ss = new ServerSocket(5000);
    System.out.println("Server waiting...");

    Socket s = ss.accept();
    System.out.println("Client connected");

    BufferedReader br = new BufferedReader(new InputStreamReader(s.getInputStream()));
    PrintWriter pw = new PrintWriter(s.getOutputStream(), true);

    BufferedReader input = new BufferedReader(new InputStreamReader(System.in));

    String msg = br.readLine();
    System.out.println("Client: " + msg);

    System.out.print("Server reply: ");
    String reply = input.readLine();
    pw.println(reply);

    s.close();
    ss.close();
}
}

```

Client.java:

```

import java.io.*;
import java.net.*;

public class Client {
    public static void main(String[] args) throws Exception {

        Socket s = new Socket("localhost", 5000);

        BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
        PrintWriter pw = new PrintWriter(s.getOutputStream(), true);
        BufferedReader in = new BufferedReader(new InputStreamReader(s.getInputStream()));

        System.out.print("Enter message: ");
        String msg = br.readLine();
        pw.println(msg);

        String reply = in.readLine();
        System.out.println("Server: " + reply);

        s.close();
    }
}

```

Output:

```

PS E:\PROJECTS\EJAVA-LAB\Assignment-5\p5> javac Server.java
PS E:\PROJECTS\EJAVA-LAB\Assignment-5\p5> javac Client.java
PS E:\PROJECTS\EJAVA-LAB\Assignment-5\p5> java Server
Server waiting...
Client connected
Client: Hey

```

Server reply: Hello

```
PS E:\PROJECTS\EJAVA-LAB\Assignment-5\p5> java Client
Enter message: Hey
Server: Hello
```

6. Write a Java TCP program to transfer a file from Server to Client.

FileServer.java:

```
import java.io.*;
import java.net.*;

public class FileServer {
    public static void main(String[] args) {
        try {
            ServerSocket ss = new ServerSocket(5000);
            System.out.println("Server started... Waiting for client");

            Socket s = ss.accept();
            System.out.println("Client connected");

            File file = new File("input.txt");

            FileInputStream fis = new FileInputStream(file);
            DataOutputStream dos = new DataOutputStream(s.getOutputStream());

            // send file name
            dos.writeUTF(file.getName());

            // send file size
            dos.writeLong(file.length());

            byte[] buffer = new byte[4096];
            int bytes;

            while ((bytes = fis.read(buffer)) != -1) {
                dos.write(buffer, 0, bytes);
            }

            fis.close();
            dos.close();
            s.close();
            ss.close();

            System.out.println("File sent successfully");

        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

FileClient.java:

```
import java.io.*;
import java.net.*;
```

```

public class FileClient {
    public static void main(String[] args) {
        try {
            Socket s = new Socket("localhost", 5000);
            System.out.println("Connected to server");

            DataInputStream dis = new DataInputStream(s.getInputStream());

            // receive file name
            String fileName = dis.readUTF();

            // receive file size
            long fileSize = dis.readLong();

            FileOutputStream fos = new FileOutputStream("received_" + fileName);

            byte[] buffer = new byte[4096];
            int bytes;
            long remaining = fileSize;

            while ((bytes = dis.read(buffer, 0, (int)Math.min(buffer.length, remaining))) > 0) {
                fos.write(buffer, 0, bytes);
                remaining -= bytes;
                if (remaining == 0) break;
            }

            fos.close();
            dis.close();
            s.close();

            System.out.println("File received successfully");

        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}

```

Output:

```

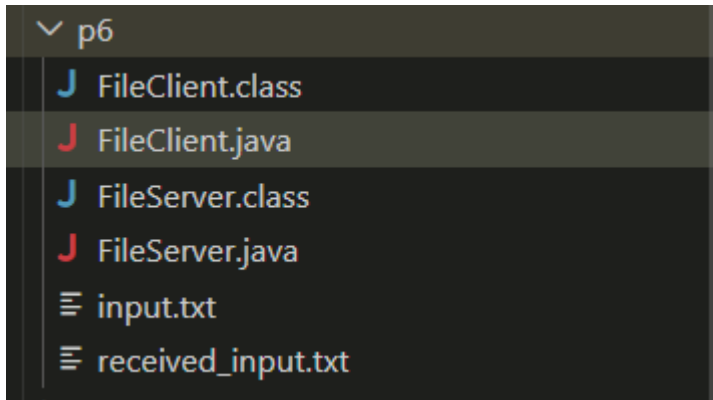
PS E:\PROJECTS\EJAVA-LAB\Assignment-5\p6> javac FileServer.java
PS E:\PROJECTS\EJAVA-LAB\Assignment-5\p6> javac FileClient.java
PS E:\PROJECTS\EJAVA-LAB\Assignment-5\p6> java FileServer
Server started... Waiting for client
Client connected
File sent successfully
PS E:\PROJECTS\EJAVA-LAB\Assignment-5\p6>

```

```

PS E:\PROJECTS\EJAVA-LAB\Assignment-5\p6> java FileClient
Connected to server
File received successfully
PS E:\PROJECTS\EJAVA-LAB\Assignment-5\p6> |

```



7. Write a Java UDP program where server echoes back whatever message the client sends

UDPServer.java

```
import java.net.*;

public class UDPServer {
    public static void main(String[] args) throws Exception {

        DatagramSocket ds = new DatagramSocket(5000);
        System.out.println("Server started...");

        byte[] buffer = new byte[1024];

        // receive
        DatagramPacket dp = new DatagramPacket(buffer, buffer.length);
        ds.receive(dp);

        String msg = new String(dp.getData(), 0, dp.getLength());
        System.out.println("Client: " + msg);

        // send back (echo)
        InetAddress ip = dp.getAddress();
        int port = dp.getPort();

        DatagramPacket dp2 = new DatagramPacket(dp.getData(), dp.getLength(), ip, port);
        ds.send(dp2);

        ds.close();
    }
}
```

UDPClient.java:

```
import java.net.*;
import java.util.Scanner;

public class UDPClient {
    public static void main(String[] args) throws Exception {

        DatagramSocket ds = new DatagramSocket();
        Scanner sc = new Scanner(System.in);
```

```

System.out.print("Enter message: ");
String msg = sc.nextLine();

byte[] buffer = msg.getBytes();

InetAddress ip = InetAddress.getByName("localhost");

// send to server
DatagramPacket dp = new DatagramPacket(buffer, buffer.length, ip, 5000);
ds.send(dp);

// receive echo
byte[] buffer2 = new byte[1024];
DatagramPacket dp2 = new DatagramPacket(buffer2, buffer2.length);
ds.receive(dp2);

String reply = new String(dp2.getData(), 0, dp2.getLength());
System.out.println("Server: " + reply);

ds.close();
}
}

```

Output:

```

PS E:\PROJECTS\EJAVA-LAB\Assignment-5\p7> javac UDPServer.java
>> javac UDPClient.java
PS E:\PROJECTS\EJAVA-LAB\Assignment-5\p7> java UDPServer
Server started...
Client: Hey its me Client
PS E:\PROJECTS\EJAVA-LAB\Assignment-5\p7> |

```

```

PS E:\PROJECTS\EJAVA-LAB\Assignment-5\p7> java UDPClient
Enter message: Hey its me Client
Server: Hey its me Client
PS E:\PROJECTS\EJAVA-LAB\Assignment-5\p7> |

```

8. Write a Java JDBC program with a menu-driven system to Insert, Display, Update, Delete and search student records.

Database Setup (MySQL):

```
CREATE DATABASE college;
```

```
USE college;
```

```
CREATE TABLE student (
  id INT PRIMARY KEY,
  name VARCHAR(50),
  marks INT
);
```

StudentCRUD.java:

```
import java.sql.*;
import java.util.Scanner;
```

```

public class StudentCRUD {

    static final String URL = "jdbc:mysql://localhost:3306/college";
    static final String USER = "root";
    static final String PASS = "root"; // change if needed

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        try {
            Class.forName("com.mysql.cj.jdbc.Driver");
            Connection con = DriverManager.getConnection(URL, USER, PASS);

            int choice;

            do {
                System.out.println("\n1.Insert 2.Display 3.Update 4.Delete 5.Search 6.Exit");
                System.out.print("Enter choice: ");
                choice = sc.nextInt();

                switch (choice) {

                    case 1: // Insert
                        System.out.print("Enter id, name, marks: ");
                        int id = sc.nextInt();
                        String name = sc.next();
                        int marks = sc.nextInt();

                        PreparedStatement ps1 = con.prepareStatement(
                            "INSERT INTO student VALUES (?, ?, ?)");
                        ps1.setInt(1, id);
                        ps1.setString(2, name);
                        ps1.setInt(3, marks);
                        ps1.executeUpdate();
                        System.out.println("Inserted");
                        break;

                    case 2: // Display
                        Statement st = con.createStatement();
                        ResultSet rs = st.executeQuery("SELECT * FROM student");

                        while (rs.next()) {
                            System.out.println(rs.getInt(1) + " "
                                + rs.getString(2) + " "
                                + rs.getInt(3));
                        }
                        break;

                    case 3: // Update
                        System.out.print("Enter id to update marks: ");
                        int uid = sc.nextInt();
                        System.out.print("Enter new marks: ");
                        int um = sc.nextInt();

                        PreparedStatement ps2 = con.prepareStatement(

```

```

        "UPDATE student SET marks=? WHERE id=?");
ps2.setInt(1, um);
ps2.setInt(2, uid);
ps2.executeUpdate();
System.out.println("Updated");
break;

case 4: // Delete
    System.out.print("Enter id to delete: ");
    int did = sc.nextInt();

    PreparedStatement ps3 = con.prepareStatement(
        "DELETE FROM student WHERE id=?");
    ps3.setInt(1, did);
    ps3.executeUpdate();
    System.out.println("Deleted");
    break;

case 5: // Search
    System.out.print("Enter id to search: ");
    int sid = sc.nextInt();

    PreparedStatement ps4 = con.prepareStatement(
        "SELECT * FROM student WHERE id=?");
    ps4.setInt(1, sid);
    ResultSet rs2 = ps4.executeQuery();

    if (rs2.next()) {
        System.out.println(rs2.getInt(1) + " "
            + rs2.getString(2) + " "
            + rs2.getInt(3));
    } else {
        System.out.println("Record not found");
    }
    break;

case 6:
    System.out.println("Exit");
    break;

default:
    System.out.println("Invalid choice");
}

} while (choice != 6);

} catch (Exception e) {
    e.printStackTrace();
}
}
}

```

Output:

```

1.Insert 2.Display 3.Update 4.Delete 5.Search 6.Exit
Enter choice: 1
Enter id, name, marks: 101 Ram 85
Inserted

```

1.Insert 2.Display 3.Update 4.Delete 5.Search 6.Exit
Enter choice: 1
Enter id, name, marks: 102 Ravi 90
Inserted

1.Insert 2.Display 3.Update 4.Delete 5.Search 6.Exit
Enter choice: 2
101 Ram 85
102 Ravi 90

1.Insert 2.Display 3.Update 4.Delete 5.Search 6.Exit
Enter choice: 3
Enter id to update marks: 101
Enter new marks: 95
Updated

1.Insert 2.Display 3.Update 4.Delete 5.Search 6.Exit
Enter choice: 2
101 Ram 95
102 Ravi 90

1.Insert 2.Display 3.Update 4.Delete 5.Search 6.Exit
Enter choice: 5
Enter id to search: 102
102 Ravi 90

1.Insert 2.Display 3.Update 4.Delete 5.Search 6.Exit
Enter choice: 4
Enter id to delete: 101
Deleted

1.Insert 2.Display 3.Update 4.Delete 5.Search 6.Exit
Enter choice: 2
102 Ravi 90

1.Insert 2.Display 3.Update 4.Delete 5.Search 6.Exit
Enter choice: 6
Exit